CENTRO UNIVERSITÁRIO ADVENTISTA DE SÃO PAULO – UNASP EC

Curso de Engenharia da Computação

Disciplina: Estrutura de Dados II

Projeto Central de Reserva – Caderno de Bordo

Alice Maria da Silva Salmazi,

Lucas Khaue Silva Teixeira,

Marcos Filipe Santos Araujo,

Pedro Augusto Alexandre da Silva.

Engenheiro Coelho - SP, 2026

Grupo criado em: 12/10/2026

Entrega final: 11/11/2026

Setor escolhido: Correios / Recebimento de encomendas do UNASP-EC.

1. OBSERVAÇÃO INICIAL

O grupo foi originalmente composto por um integrante que trancou o curso e foi substituído posteriormente pelo Lucas (por isso o nome dele não aparece nas primeiras etapas). A troca ocorreu entre E1 e E2. O setor foi escolhido por ser ponto central de fluxo diário e por ter fila real de atendimento, adequada aos cinco achados do projeto.

2. CRONOLOGIA DAS SESSÕES

2.1. SESSÃO I (12/10/2026)

Quem: grupo completo;

Feito: criação do grupo; definição do setor (Correios); primeiras decisões sobre escopo e modelagem de dados.

Decidido: usar modelagem JSON/Dicionário para encomenda; mapear Recurso (prateleira) × Reserva (encomenda).

2.2. SESSÃO II (13/10/2026)

Quem: Marcos e Pedro (presencial); Alice (organização).

Feito: ida ao setor; conversa com a coordenadora do Correios para autorização da coleta; levantamento do inventário de prateleiras.

Decidido: acompanhar duas janelas de observação.

2.3. SESSÃO III (16/10/2026 - Tarde e 20/10/2026 - Manhã)

Quem: Marcos e Pedro

Feito: observação das duas janelas no ponto de atendimento.

Não funcionou: o volume de pedidos observado presencialmente foi insuficiente para a base mínima de 60 pedidos.

Decidido: solicitar ao setor uma planilha retrospectiva, sem identificação de alunos, para completar a série. Adaptação declarada no relatório (mistura de observação direta + registro retrospectivo).

2.4. SESSÃO IV (14/10/2026 - E1)

Quem: grupo completo.

Feito: entrega do escopo e base de campo.

Não funcionou: entrega no limite do prazo (reflexo da troca de integrante líder ocorrida na mesma semana).

Decidido: reorganização interna das tarefas por módulo.

2.5. SESSÃO V (28/10/2026 - E2 e 04/11/2026 - E3)

Quem: Lucas (tads.py); Marcos (arvore.py e arvoreb.py); Pedro (heap.py); Alice (base carregada + revisão de arquivos de entrega).

Feito: E2 e E3 entregues juntos (decisão consciente - ver observação abaixo). Implementação de tads.py, arvore.py e heap.py.

Não funcionou: entregar E2 e E3 separados não foi viável após a troca de integrante; acumulou trabalho.

Decidido: entregar E2+E3 juntos no checkpoint de 04/11, com base carregada e 'coerente()' passando nos três módulos.

Observação do grupo: Acabamos entregando as duas etapas juntas no Módulo 3 já que não conseguimos nos organizar e pela troca de um integrante (líder do grupo).

2.6. SESSÃO VI (05 a 10/11/2026 - preparação para E4)

Quem: Alice (revisão de aderência ao PDF + organização da base); Lucas, Marcos e Pedro (revisão de código); Marcos (construção relatório Achados).

Feito: revisão cruzada dos arquivos para alinhar com a documentação do professor.

Correções registradas na seção 3 deste caderno (Registro de uso de IA). Ao final da sessão, os 4 achados A1–A4 estavam sendo medidos no programa com 'coerente() = True' em todas as estruturas, e a trilha B já retornava BFS × Dijkstra com números reais.

Não funcionou: a base tinha apenas 2 datas distintas em 'dataChegada', o que impedia o A1 de mostrar a degeneração da BST (corrigido ao usar 'numEtiqueta') como chave ordenada (ver bloco 5 da seção 3). Também surgiu um bug de regressão no Heap ao integrar o 'MinHeap(chave=...)'.

Decidido: priorizar correções que desbloqueiam os cinco achados (A1, A2, A3, A4, A5).

2.7. SESSÃO VII (11/11/2026 - pacote congelado)

Quem: grupo completo.

Feito: entrega final (main.py, cadastro.py, tads.py, arvore.py, arvoreb.py, heap.py, relatorio.py, trajetoria.py, inventario.json, pedidos.csv, trechos.csv, relatorio.txt, relatorio.pdf, caderno_de_bordo.pdf, revisão por pares).

Não funcionou: no fechamento, o Git rejeitou o push porque o colega havia commitado 'Heap.py' (maiúsculo) em paralelo, conflitando com nossa migração para 'heap.py' (minúsculo). Resolvido com merge preservando histórico do colega e mantendo 'heap.py' minúsculo (registrado na seção 4, caso F).

Decidido: congelamento do código; foco na defesa.

3. DIVISÃO DE RESPONSABILIDADES

Resumo por módulo (informado pelo grupo):

- ✓ tads.py → Lucas
- ✓ arvore.py → Marcos
- ✓ arvoreb.py → Marcos (adaptação posterior)
- ✓ Achados.pdf → Marcos
- ✓ heap.py → Pedro
- ✓ base carregada → Alice

Tabela por etapa:

Integrante	*E1*	*E2*	*E3*	*E4*
Alice	Estrutura JSON	Base de dados	Revisão	Revisão + Caderno de Bordo
Lucas	-	tads.py	tads.py	Revisão
Marcos	Coleta	arvore.py	arvoreb.py	Revisão + Achados
Pedro	Coleta	heap.py	heap.py	Revisão

4. REGISTRO DE USO DE FERRAMENTAS DE IA

4.1. BLOCO I (Modelagem de dados - E1, 12-13/10/2026)

O que pedimos:

- ✓ Estruturar em formato JSON/Dicionário as entidades do sistema (encomenda + recurso), e definir as listas de domínio (embalagem,

transportadoras, categorias de público, status), além de uma política de prioridade baseada no tipo de solicitante.

O que a IA devolveu:

- ✓ Estrutura JSON inicial (numero_etiqueta, numero_cadastro, embalagem, transportadora, destinatario{...}, status, data_chegada, data_retirada, retirado_por, observacoes).
- ✓ Separação Recurso × Reserva (codigo, tipo, capacidade, local).
- ✓ Listas completas de embalagem, transportadoras, categoria de público e status.
- ✓ Escala de prioridade inicial (aula regular 1 / evento 2 / projeto 3 / uso livre 4).

O que o grupo alterou e por quê:

- ✓ Substituímos os nomes de campo do modelo genérico pelos status reais do domínio do Correios: ENTREGUE / AGUARDANDO (status do arquivo de listas). Motivo: o nosso setor não tem o ciclo "reservado → disponível" — tem "chegou → retirado". A modelagem precisava refletir o domínio real.
- ✓ Rejeitamos a escala de prioridade genérica (aula regular, evento institucional, projeto, uso livre) porque é da Central de Reservas original, não do Correios. Definimos uma escala nova por categoria de público:
  - Prioridade 1: DEPARTAMENTOS, CANTINA, REITORIA, JURÍDICO
  - Prioridade 2: ALUNO INTERNO
  - Prioridade 3: FUNCIONÁRIO INTERNO / RESIDENTE / EXTERNO
  - Prioridade 4: ALUNO EXTERNO, EXTERNO

  Motivo: a documentação diz que 'solicitante_tipo' e a prioridade não são medição: são política. O grupo define as classes e justifica por escrito. A escala original não fazia sentido no nosso setor.
- ✓ Adicionamos `solicitantePermitido` em cada recurso (prateleira) com a lista de categorias que ela aceita. Motivo: a política de prioridade só existe se o sistema também impedir alocação cruzada (aluno externo não pode ocupar prateleira de prioridade 1). Sem essa validação, a política era decoração.

4.2. BLOCO II (Arquitetura do menu e persistência, E1→E2, 13/10-28/10/2026)

O que pedimos:

- ✓ Organizar o projeto em main.py (menu numerado) e cadastro.py (persistência + validações), com leitura de pedidos.csv e inventario.json. Implementar as opções 2 (gestão de encomendas) e 3 (gestão de prateleiras) com a lógica de vincular etiqueta à prateleira respeitando a capacidade.

O que a IA devolveu:

- ✓ Classe unificada Cadastro com carregar_dados() usando csv.DictReader convertendo linhas em dicionários aninhados.
- ✓ Esqueleto de menu com while/try-except.
- ✓ Método inserir_item_na_prateleira(codigo_recurso, num_etiqueta) que verificava apenas capacidade.

O que o grupo alterou e por quê:

- ✓ Adicionamos a validação de prioridade dentro de inserir_item_na_prateleira — a versão da IA só verificava capacidade. Motivo: de nada adianta declarar política de prioridade em cadastro.py se o sistema aceita alocar aluno externo (prioridade 4) na prateleira A (prioridade 1). A validação cruzada é o que dá sentido à política.
- ✓ Invertemos a lógica das opções 2 e 3 em relação ao que a IA propôs. A IA originalmente colocava "cadastrar prateleira" na opção 2. Movemos para a opção 3, porque a opção 2 é a operação DIÁRIA (cadastrar encomenda + alocar + editar + remover) e a opção 3 é a configuração de INFRAESTRUTURA (criar prateleiras). Motivo: refletir o fluxo real do setor (cadastra-se encomenda todo dia, mas prateleira é raro).
- ✓ Removemos a chamada dupla de gerenciador.cadastrar("pedido", ...) que a IA havia gerado em refatoração anterior. Motivo: a segunda chamada lançava ValueError DEPOIS do cadastro ter dado certo - o usuário via erro vermelho mesmo com sucesso. Registrado como caso C na seção 4.

4.3. BLOCO III (Integração das estruturas do grupo, 28/10-04/11/2026)

O que pedimos:

- ✓ Integrar tads.py (Fila/Pilha), arvore.py (árvore) e heap.py (Min-Heap + heapsort) às opções avançadas do menu (4, 6, 8, 9, 10, 11), exibindo contadores ao final (nós visitados, operações etc.).

O que a IA devolveu:

- ✓ Mapeamento de imports e chamadas.
- ✓ Opção 4 (agenda) implementada como varredura linear apenas.
- ✓ Opção 6 (lista de espera) usando MinHeap e exibindo ‘operacoes’.
- ✓ Opção 8 (árvore B) construindo 1 árvore por vez.

O que o grupo alterou e por quê:

- ✓ Exigimos heap.py em minúsculo (não Heap.py). Motivo: a documentação nomeia o módulo em minúsculo (heap.py). Nome errado do arquivo pode ser interpretado como não-aderência ao escopo.

- ✓ Exigimos propriedade de tamanho em vez de len(lista) direto dentro do Heap. Motivo: o professor exige que se use a TAD do grupo, não lista crua do Python por baixo do capô.
- ✓ Separamos os contadores em 'comparacoes' e 'trocas' (a IA usava um único 'operacoes'). Motivo: a documentação pede contadores separados de comparações e trocas.
- ✓ Exigimos 'coerente()' na MinHeap. Motivo: (checklist) exige coerente() implementada e passando em arvore.py, arvoreb.py e heap.py.
- ✓ Exigimos 'k_menores_com_heap(itens, k)' como função separada. Motivo: a documentação exige essa função específica; sem ela, o A4 do relatório não tem número.
- ✓ Opção 4 passou a oferecer BST / AVL / varredura. Motivo: a documentação pede as três estruturas no menu; a IA propôs só varredura, o que não gera o número do A2.
- ✓ Opção 8 passou a construir 3 árvores B com m=3, m=5 e m=51, e passou a comparar com a AVL na mesma base. Motivo: a documentação exige pelo menos três ordens E a comparação com AVL.
- ✓ Opção 10 - ver Bloco 4 abaixo (tem adaptação de domínio).

4.4. BLOCO IV (Consulta, undo (Pilha) e Trilha B - 05-10/11/2026)

O que pedimos:

- ✓ Implementar opções 5 (consulta), 7 (desfazer com Pilha), 9 (relatório consolidado), 10 (Trilha B — deslocamento no campus) e 11 (gravar relatório).

O que a IA devolveu:

- ✓ Opção 5 (varredura no inventário).
- ✓ Opção 7 (pilha de histórico — mas só "cadastrar pedido" empilhava e só "cadastrar" desfazia).
- ✓ Opção 9 (só media construção do MinHeap, sem A1/A2/A4).
- ✓ Opção 10 (verificava capacidade excedida, não deslocamento).
- ✓ Opção 11 (gerava um stub com 3 linhas).

O que o grupo alterou e por quê:

- ✓ Opção 7 reescrita: TODA mutação (cadastrar/editar/remover pedido, cadastrar/remover prateleira, alocar) passou a empilhar, com snapshot do estado anterior para reverter. Motivo: a documentação diz "histórico de operações com desfazer" e a IA original só fazia 1 das 6 mutações, o que não é "histórico", é decoração.

- ✓ Opção 9 passou a delegar para relatorio.py (novo módulo), gerando as tabelas de A1, A2, A3 e A4. Motivo: a documentação pede "relatório de desempenho das estruturas"; exige tabela por achado.
- ✓ Opção 10 reescrita como Trilha B (grafo do campus). A IA inicialmente propôs medir distância entre PRATELEIRAS (o grupo corrigiu: a Trilha B fala do GRAFO DO CAMPUS (Faculdade, Prédio Central, Reitoria etc.), não das prateleiras. Depois o grupo verificou que o Correios fica dentro da Faculdade, então as prateleiras são sub-nós internos irrelevantes para conflito de deslocamento. A malha final tem só os prédios do campus.
- ✓ Opção 10 - declaração de adaptação: a documentação pede conflito entre reservas consecutivas em locais distintos. No nosso domínio (Correios), todas as prateleiras estão na Faculdade, então "conflito entre retiradas" não se aplica. Mantivemos a Trilha B na parte de ALGORITMOS (BFS com Fila × Dijkstra com Heap), que é o que o Anexo B destaca. A decisão de não forçar conflito artificial está registrada no A5 do relatório.
- ✓ Opção 11 passou a gravar o MESMO conteúdo da opção 9 em relatorio.txt. Motivo: (checklist) exige "relatorio.txt gerado pelo próprio programa"; a IA original gerava algo desconectado do que a opção 9 mede.

4.5. BLOCO V (Correção do A1 e do A2 - 10/11/2026)

Contexto:

- ✓ A opção 9 rodou pela primeira vez com as estruturas prontas, e os quatro achados saíram com números, mas dois deles não demonstravam o fenômeno que é pedido.

O que a IA devolveu:

- ✓ A1 usando ‘dataChegada’ como chave. Como a base só tinha 2 datas distintas, BST e AVL saíam com altura 2 (idênticas) – sem contraste, sem achado.
- ✓ A2 com quartis calculados por `sorted()` das datas.

O que o grupo alterou e por quê:

- ✓ A1 passou a usar ‘numEtiqueta’ como chave (60 valores únicos, inseridos em ordem crescente via insertion sort manual). Motivo: reproduzir o cenário do §10 ("a BST degenera quando recebe chaves em ordem") sem alterar a base. O resultado: BST altura 60 e AVL altura 6. A explicação (BST com altura 60 porque cada nova etiqueta é filho direito do anterior; AVL com altura 6 porque rebalanceia com rotações) está no A1 do relatório.
- ✓ A2 passou a calcular faixas por quartis de VOLUME de pedidos (não por datas únicas) + uma faixa extra com o dia de maior frequência. Algoritmo: insertion sort manual das chaves (sem ‘sorted’) + contagem de frequência

por ‘dict’ (permite dict para cadastro/contagem, não como índice ordenado). Motivo: respeitar e cobrir ("incluindo o dia mais movimentado") sem depender de hardcode de datas.

- ✓ Removemos o ‘sorted()‘ que havia ficado na primeira versão do A2. Motivo: na documentação – ‘sorted()’ só é permitido em ‘verificar_contra_lista’.
- ✓ ‘medir_bst_avl’ foi ajustado para capturar ‘comparacoes’ ANTES das buscas (‘comp_ins = arv.comparacoes’). Motivo: sem isso, o contador ‘comparacoes_insercao’ misturava comparações de inserção com comparações das 4 buscas do A1 - o número ficava inflado e não batia com o Anexo C.

4.6. BLOCO 6 (Integração com o Git do grupo - 11/11/2026)

Contexto:

- ✓ O grupo manteve um único repositório no GitHub (alicesalmazi/Correios). Cada integrante puxava o estado do dia, trabalhava localmente e commitava de volta.

O que a IA devolveu:

- ✓ Estratégia de ‘git push –force’ para resolver conflito de push.

O que o grupo alterou e por quê:

- ✓ Rejeitamos o ‘--force’ porque apagaria o commit do colega no histórico do GitHub - mesmo tendo copiado o conteúdo dele localmente. Motivo: a documentação exige que o histórico mostre quem fez o quê; force- push esconde a contribuição alheia e pode custar ponto de processo. A solução usada foi ‘git merge -X ours origin/main’ (ou ‘git rebase’ quando não havia conflito), preservando os commits do colega no histórico e mantendo o conteúdo local correto.
- ✓ No conflito específico ‘Heap.py deleted in remote, modified in HEAD’, resolvemos mantendo ‘heap.py’ minúsculo. Motivo: a documentação nomeia o módulo em minúsculo; o colega havia deletado o ‘Heap.py’ maiúsculo (na migração), mas o Git marcou conflito porque nossa cópia local ainda tinha o arquivo antigo. Registrado como caso F na seção 4.

5. UM DEFEITO QUE NÓS ENCONTRAMOS

5.1. CASO PRINCIPAL - Invariante quebrado entre CSV e JSON

Defeito:

- ✓ Remover um pedido pelo menu (opção 2 → remover) apagava a linha do pedidos.csv, MAS o número da etiqueta continuava listado e itemVinculado de alguma prateleira do inventario.json. O invariante "toda etiqueta em

itemVinculado existe em pedidos.csv ficava quebrado após qualquer remoção.

Teste que revelou:

- ✓ Rodamos a opção 5 (consulta de disponibilidade) imediatamente após remover a etiqueta ETQ-2026-0003 pelo menu. A prateleira PRAT-P3-01 continuou mostrando "Itens Alocados: [ETQ-2026-0003]" mesmo após a remoção. Confirmamos o mesmo com contagem cruzada: ‘pedidos.csv’ tinha 60 linhas e ‘inventario.json’ referenciava 1 etiqueta a mais - divergência de 1.

Correção:

- ✓ ‘remover(tipo="pedido", ...)’ passou a chamar internamente ‘remover_item_da_prateleira(identificador)’ após apagar a linha do CSV. O invariante voltou a valer.

Lição aprendida:

- ✓ O defeito não dava erro - o programa "funcionava", o CSV tinha 60 linhas, o JSON tinha 61. Só foi visto quando olhamos os dois arquivos ao mesmo tempo. É exatamente o tipo de defeito que a documentação descreve: "aparece como invariante quebrado, não como exceção."

5.2. COMPLEMENTO - Regressão no MinHeap (‘dict < dict’)

Defeito:

- ✓ Ao ganhar ‘MinHeap(chave=None)’ para ser reutilizado no Dijkstra (trajetoria.py), o default passou a ser ‘lambda item: item’ (identidade). A opção 6 (lista de espera) chamava ‘MinHeap()’ sem argumento e passou a comparar dicionários diretamente - Python não compara dict com ‘<’, então o primeiro ‘_subir’ estourava com ‘TypeError: '<' not supported between instances of 'dict' and 'dict’’. Além disso, ‘main.py’ ainda referenciava ‘h.operacoes’, atributo que o novo heap.py removeu (substituído por ‘comparacoes’ + ‘trocas’).

Teste que revelou:

- ✓ Rodamos ‘python main.py’ → opção 6 → ‘[2] Min-Heap’. O TypeError apareceu na primeira comparação feita pelo ‘_subir’.

Correção:

- ✓ Default do ‘MinHeap’ voltou a ser ‘chave_da_encomenda’,

preservando compatibilidade com quem chama ‘MinHeap()’ sem

argumento;

- ✓ ‘main.py’ trocou ‘h.operacoes’ por ‘h.comparacoes’ e ‘h.trocas’;

- ✓ Adicionamos a classe ‘Lista’ em ‘tads.py’ - o ‘heap.py’ importava ‘from tads import Lista’, mas a classe não existia ainda (segundo defeito que só foi descoberto por que o primeiro fez o programa abrir a opção 6).

Lição aprendida:

- ✓ A mudança que corrigiu um problema (separar comparações e trocas; chave opcional para o Dijkstra) introduziu outro (perda do default compatível). Adicionamos esse teste ao roteiro de aceitação de ‘heap.py’ (executa opção 6 → Min-Heap e confere ‘coerente() == True’).

6. OBSERVAÇÕES FINAIS E ADAPTAÇÕES DECLARADAS

O grupo fez quatro adaptações conscientes ao escopo do caderno. Elas estão registradas aqui e no relatório técnico (A5), porque a documentação permite mudanças de política desde que justificadas:

- ✓ Base de campo mistura observação direta (16/10 tarde + 20/10 manhã) e registro retrospectivo (planilha do setor). Declarado no relatório por exigência da documentação.
- ✓ Lista de espera foi adaptada: "recurso disputado" é a prateleira, não o horário. AGUARDANDO não entra na fila (ainda não chegou); só ENTREGUE sem prateleira disputa vaga.
- ✓ Trilha B mantida na parte de algoritmos (BFS × Dijkstra no grafo do campus), mas a detecção de conflito entre retiradas consecutivas não se aplica porque todas as prateleiras do Correios ficam no mesmo prédio (Faculdade).
- ✓ No A1, usamos ‘numEtiqueta’ como chave em vez de ‘dataChegada’. A documentação exige demonstrar a degeneração da BST com carga em ordem crescente; a base tem apenas 2 datas distintas em ‘dataChegada’, o que tornaria o fenômeno invisível (BST e AVL com altura 2 ambas). Com ‘numEtiqueta’ (60 valores únicos) inseridos em ordem crescente, a BST atinge altura 60 e a AVL altura 6 - reproduzindo o fenômeno sem alterar a base.